

AI Multi-Step Prompt Workflow Assessment

Name : B.Sai Ganesh

Register Number : 99230040272

1. Scenario

Scenario:

Transforming Raw Technical Notes into an API Quick-Start Guide

Goal:

Take messy technical notes about a student portal API and transform them into a clear quick-start guide, followed by a self-audit and refined final version.

Workflow:

- Step 1 — Extraction & Planning: Raw notes → documentation outline
- Step 2 — Generation/Drafting: Outline → first API quick-start guide
- Step 3 — Evaluation/Refinement: Draft → checklist audit → corrected final guide

2. Step 1 — Extraction & Planning

Prompt 1 — Exact Prompt:

Act as an expert technical documentation writer.

I am going to give you raw, messy notes about a student portal API.

Your task:

1. Extract the key API capabilities from the notes.
2. Organize the information into a logical quick-start guide outline.
3. Identify the important endpoint, request, response, authentication, and usage information that is actually supported by the notes.
4. Do not invent technical details that are not present in the notes.

The outline should include:

- A short introduction
- API capabilities
- Endpoint information
- Request and response information
- Authentication or access requirements, if supported by the notes
- A simple usage flow
- Common considerations or limitations, if supported by the notes
- A short conclusion

Raw Notes:

The student portal API allows applications to retrieve student profile information, retrieve course

information, and view student progress. The profile endpoint uses GET and requires a student ID. The course endpoint uses GET and accepts a course ID. The progress endpoint uses GET and requires both a student ID and course ID. API access requires an API key. Responses are returned in JSON format. Applications should check the response status before processing the returned data. The API is intended to help applications display student and course information in a structured way.

Intermediate AI Response — Step 1:

Structured API Quick-Start Guide Outline

1. Introduction

- Explain that the API provides structured access to student, course, and progress information.
- Mention that responses are returned in JSON format.

2. API Capabilities

- Retrieve student profile information.
- Retrieve course information.
- View student progress.

3. Authentication and Access

- API access requires an API key.
- Applications should use the API key when accessing the API.

4. Available Endpoints

- Student Profile: GET; requires a student ID.
- Course Information: GET; accepts a course ID.
- Student Progress: GET; requires both a student ID and course ID.

5. Request and Response Information

- Requests use the GET method for the documented endpoints.
- Responses are returned in JSON format.
- Applications should check the response status before processing returned data.

6. Basic Usage Flow

- Obtain the required API key.
- Select the required endpoint.
- Provide the required identifier(s).
- Send the GET request.
- Check the response status.
- Process the JSON response.

7. Conclusion

- Summarize how the API can provide structured student, course, and progress information to applications.

3. Step 2 — Generation/Drafting

Prompt 2 — Exact Prompt:

Excellent. Now, using only the outline you just generated, write the first draft of a clear API quick-start guide.

Follow these constraints strictly:

- Use a professional and beginner-friendly technical writing style.
- Do not invent endpoint URLs, parameter names, JSON fields, HTTP status codes, or authentication procedures that were not provided in the raw notes or outline.
- Clearly explain all three API capabilities.
- Clearly identify the documented HTTP method and required identifiers.
- Explain the API key requirement.
- Explain that responses are returned in JSON format.
- Include a simple step-by-step usage flow.
- Use short sections and concise paragraphs.
- Include a placeholder for an API workflow diagram.

After writing the guide, briefly state the approximate word count.

Intermediate AI Response — Step 2:

Student Portal API Quick-Start Guide

Introduction

The Student Portal API provides applications with structured access to student profile information, course information, and student progress. The API uses GET requests for the documented endpoints and returns responses in JSON format.

[Image Placeholder: API request and response workflow]

API Capabilities

The API supports three main capabilities:

1. Retrieve student profile information.
2. Retrieve course information.
3. View student progress.

Authentication

API access requires an API key. Applications should provide the required API key when accessing the API.

Available Endpoints

Student Profile

- Method: GET
- Required information: Student ID
- Purpose: Retrieve student profile information.

Course Information

- Method: GET
- Required information: Course ID
- Purpose: Retrieve course information.

Student Progress

- Method: GET
- Required information: Student ID and Course ID
- Purpose: View student progress.

Basic Usage Flow

1. Obtain the required API key.
2. Select the endpoint that provides the information you need.
3. Provide the required identifier or identifiers.
4. Send the GET request.
5. Check the response status.
6. Process the returned JSON response.

Important Considerations

Make sure the required identifier information is available before sending a request. Applications should check the response status before processing the returned data.

Conclusion

The Student Portal API provides a structured way for applications to retrieve student profiles, course information, and student progress. By using the documented GET endpoints and processing the JSON responses, applications can display this information in a structured way.

Approximate word count: 285 words.

4. Step 3 — Critique & Refinement

Prompt 3 — Exact Prompt:

Before finalizing the API quick-start guide, act as a strict senior technical editor.

Review the draft you just wrote against this checklist:

1. Does the guide clearly explain all 3 API capabilities from Step 1?
2. Are the documented HTTP methods correct?
3. Are the required identifiers for each endpoint clearly stated?
4. Is the API key requirement included?
5. Is the JSON response format mentioned?
6. Does the guide avoid inventing endpoint URLs, parameters, JSON fields, status codes, or authentication details?
7. Does it include the basic usage flow?
8. Does it include the API workflow diagram placeholder?
9. Is the writing professional and beginner-friendly?
10. Are the sections concise and easy to follow?

First, provide a concise bullet-point critique identifying any issues or risks.

Second, produce the final corrected version of the API quick-start guide, fixing any issues while preserving

only information supported by the raw notes and Step 1 outline.

At the end, provide the approximate word count of the final version.

Intermediate AI Response — Step 3 Critique:

Senior Technical Editor Critique

- API capabilities: Satisfied. All three capabilities are clearly explained.
- HTTP methods: Satisfied. The documented endpoints are correctly identified as GET.
- Required identifiers: Satisfied. Student ID and/or Course ID requirements are stated for each endpoint.
- API key requirement: Satisfied.
- JSON response format: Satisfied.
- Avoidance of invented details: Satisfied. No endpoint URLs, parameter names, JSON fields, status codes, or unsupported authentication procedures were added.
- Basic usage flow: Satisfied.
- Diagram placeholder: Satisfied.
- Writing style: Satisfied. The guide is professional and beginner-friendly.
- Organization: Satisfied. The sections are concise and easy to follow.

Overall: The draft meets the checklist. Minor wording refinements can improve consistency and clarity.

Final Corrected Version — Step 3:

Student Portal API Quick-Start Guide

Introduction

The Student Portal API provides applications with structured access to student profile information, course information, and student progress. The documented endpoints use GET requests and return responses in JSON format.

[Image Placeholder: API request and response workflow]

API Capabilities

The API supports three main capabilities:

1. Retrieve student profile information.
2. Retrieve course information.
3. View student progress.

Authentication

API access requires an API key. Applications should use the required API key when accessing the API.

Available Endpoints

Student Profile

- Method: GET
- Required information: Student ID
- Purpose: Retrieve student profile information.

Course Information

- Method: GET
- Required information: Course ID
- Purpose: Retrieve course information.

Student Progress

- Method: GET
- Required information: Student ID and Course ID
- Purpose: View student progress.

Basic Usage Flow

1. Obtain the required API key.
2. Select the endpoint that provides the required information.
3. Provide the required identifier or identifiers.
4. Send the GET request.
5. Check the response status.
6. Process the returned JSON response.

Important Considerations

Ensure that the required identifier information is available before sending a request. Applications should check the response status before processing the returned data.

Conclusion

The Student Portal API provides a structured way for applications to retrieve student profiles, course information, and student progress. Following the documented GET endpoints and processing the JSON responses allows applications to display this information in a structured way.

Approximate word count: 275 words.

5. Submission Checklist

- Exact Prompt 1, Prompt 2, and Prompt 3 text
- Intermediate AI response from each step
- Final corrected output from Step 3
- Screenshot or chat link showing all three steps in the same live chat session

6. Live Conversation Evidence

<https://chatgpt.com/share/6a9ec6a3-c3a4-83e8-b300-368a402ab822?ogimg=plain>